

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS:

Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.
- (b) Design input fields for email and phone number with appropriate HTML attributes.
- (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
- (d) Develop a mechanism to display **real-time error messages** for invalid inputs.
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- (a) Analyze the CSS properties responsible for layout overlapping issues.
- (b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
- (c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
- (d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- (e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

I K PHANI SRIDHAR YOGI 192311091 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

K-P-Sridhar

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentati on	10	Clear, well- structured documentation with diagrams, code, and results	Organized documentati on with necessary details	Basic documentati on with missing details	Poor or incomplete documentati on
-------------------	----	--	---	---	--

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using CSS Flexbox or Grid, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to different screen sizes (mobile, tablet, desktop) using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Sure. Here is a **short assignment version** that still covers all the rubric points.

Aim

To design a responsive retail webpage using **CSS Flexbox, Grid, and Media Queries** that works properly on mobile, tablet, and desktop devices.

(a) Reasons for Lack of Responsiveness

- Use of fixed widths and heights.
- No CSS media queries.
- Improper use of margins and padding.
- Images are not responsive.
- Navigation does not adapt to small screens.
- Lack of Flexbox or Grid layout.

- Missing responsive viewport meta tag.

(b) Responsive Webpage Design

The webpage contains:

- **Header** – Logo, search bar, and cart.
- **Navigation** – Home, Products, Offers, About, Contact.
- **Content** – Responsive product cards using CSS Grid.
- **Footer** – Company and contact information.

HTML

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Responsive Retail Store</title>
```

```
  <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
  <header>
```

```
    <h2>RetailHub</h2>
```

```
    <input type="text" placeholder="Search Products">
```

```
    <span>🛒 Cart</span>
```

```
  </header>
```

```
  <nav>
```

```
    <a href="#">Home</a>
```

```
    <a href="#">Products</a>
```

```
<a href="#">Offers</a>
<a href="#">Contact</a>
</nav>
```

```
<main>
  <h1>Our Products</h1>

  <section class="products">
    <div>Product 1</div>
    <div>Product 2</div>
    <div>Product 3</div>
    <div>Product 4</div>
  </section>
</main>
```

```
<footer>
  <p>© 2026 RetailHub. All Rights Reserved.</p>
</footer>
```

```
</body>
```

```
</html>
```

CSS

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
```

```
}
```

```
body {  
  font-family: Arial;  
  background: #f5f5f5;  
}
```

```
header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  gap: 20px;  
  padding: 20px;  
  background: #222;  
  color: white;  
}
```

```
header input {  
  width: 50%;  
  padding: 10px;  
}
```

```
nav {  
  display: flex;  
  justify-content: center;  
  gap: 25px;
```

```
padding: 15px;  
background: white;  
}
```

```
nav a {  
    text-decoration: none;  
    color: black;  
}
```

```
main {  
    width: 90%;  
    max-width: 1200px;  
    margin: auto;  
    padding: 30px 0;  
    text-align: center;  
}
```

```
.products {  
    display: grid;  
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
    gap: 20px;  
    margin-top: 20px;  
}
```

```
.products div {  
    padding: 50px 20px;
```



```
background: white;
border-radius: 8px;
}
```

```
footer {
padding: 25px;
text-align: center;
background: #222;
color: white;
}
```

```
/* Tablet */
```

```
@media (max-width: 768px) {
header {
flex-wrap: wrap;
}
```

```
header input {
width: 100%;
}
```

```
.products {
grid-template-columns: repeat(2, 1fr);
}
}
```

```

/* Mobile */
@media (max-width: 480px) {
  header {
    flex-direction: column;
  }

  nav {
    flex-direction: column;
    align-items: center;
  }

  .products {
    grid-template-columns: 1fr;
  }
}

```

(c) Responsive Adaptation

Device Layout

Desktop Multiple product columns

Tablet Two product columns

Mobile One product column

Media queries automatically adjust the header, navigation, and product layout according to screen size.

(d) Spacing and Alignment

- **Flexbox** is used for header and navigation alignment.
- **CSS Grid** is used for product cards.
- `gap` provides consistent spacing.

- padding and margin improve visual appearance.
- justify-content and align-items distribute elements properly.
- max-width prevents excessive content stretching.

(e) Evaluation and Improvements

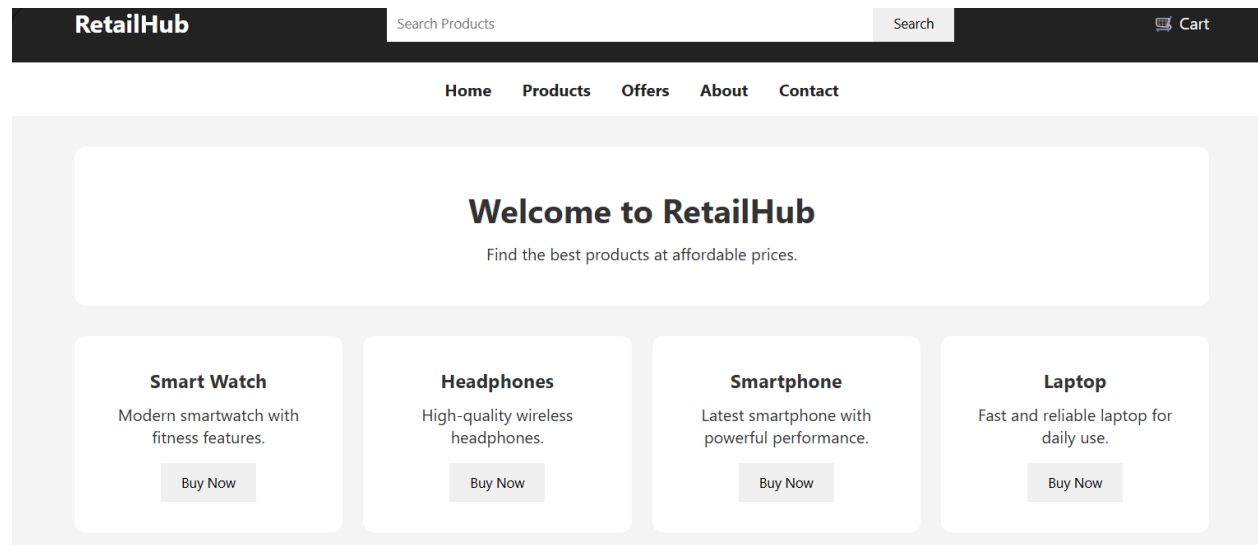
The responsive design successfully prevents content overlap and maintains proper alignment on different devices. The use of Flexbox, Grid, and media queries improves usability and visual consistency.

Further improvements:

- Add a hamburger menu for mobile.
- Use responsive images.
- Improve accessibility.
- Test on real mobile and tablet devices.

Result

The responsive retail webpage was successfully implemented using **HTML, CSS Flexbox, CSS Grid, and Media Queries**. The webpage adapts effectively to **mobile, tablet, and desktop** screen sizes with proper alignment, spacing, and distribution.



Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.**
- (b) Design input fields for email and phone number with appropriate HTML attributes.**
- (c) Implement JavaScript validation using Regular Expressions for both email and phone number.**
- (d) Develop a mechanism to display real-time error messages for invalid inputs.**
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.**

Aim

To validate email and phone number inputs using **HTML attributes, JavaScript, and Regular Expressions.**

(a) Issues in Existing Form Validation

- Invalid email addresses are accepted.
- Incorrect phone numbers can be submitted.
- No real-time error messages are displayed.
- Users receive feedback only after submitting the form.
- Client-side validation may be incomplete or missing.

(b) HTML Input Fields

```
<form id="registrationForm">
```

```
<label>Email:</label>
```

```
<input type="email" id="email" required
      placeholder="example@gmail.com">
<small id="emailError"></small>

<label>Phone Number:</label>
<input type="tel" id="phone" required
      placeholder="9876543210"
      pattern="[0-9]{10}">
<small id="phoneError"></small>

<button type="submit">Register</button>

</form>
```

```
<script src="script.js"></script>
```

(c) JavaScript Regular Expression Validation

```
const email = document.getElementById("email");
const phone = document.getElementById("phone");

const emailError = document.getElementById("emailError");
const phoneError = document.getElementById("phoneError");

const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const phonePattern = /^[0-9]{10}$/;

email.addEventListener("input", function() {
```

```
if (!emailPattern.test(email.value)) {  
    emailError.textContent = "Enter a valid email address";  
} else {  
    emailError.textContent = "";  
}  
});  
  
phone.addEventListener("input", function() {  
    if (!phonePattern.test(phone.value)) {  
        phoneError.textContent = "Enter a valid 10-digit phone number";  
    } else {  
        phoneError.textContent = "";  
    }  
});
```

(d) Real-Time Error Messages

The input event checks the data whenever the user types.

Example:

Email: abc@

Error: Enter a valid email address

Phone: 12345

Error: Enter a valid 10-digit phone number

When the user enters valid data, the error message automatically disappears.

(e) Evaluation and Enhancements

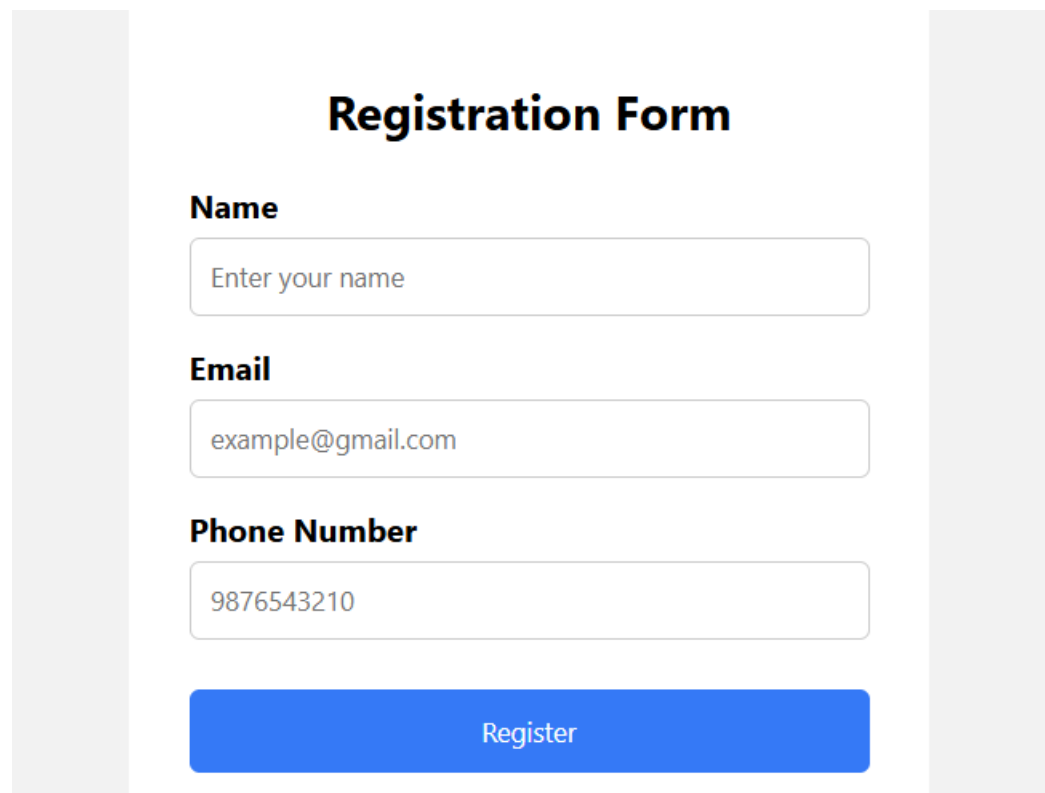
The validation system successfully identifies invalid email and phone number inputs before form submission. Regular Expressions ensure that the entered data follows the required format, while real-time messages provide immediate feedback.

Further Enhancements

- Add password strength validation.
- Validate all required fields.
- Use server-side validation for security.
- Sanitize user input to prevent attacks.
- Provide clear and user-friendly error messages.
- Prevent form submission when invalid data is entered.

Result

The registration form was successfully validated using **HTML5 attributes, JavaScript, and Regular Expressions**. Real-time error messages help users correct invalid email and phone number inputs immediately, improving data accuracy and user experience.

A registration form with a title 'Registration Form' in bold black text. Below the title are three input fields: 'Name' with placeholder 'Enter your name', 'Email' with placeholder 'example@gmail.com', and 'Phone Number' with placeholder '9876543210'. At the bottom is a blue 'Register' button. The form is centered between two vertical gray bars.

Registration Form

Name

Email

Phone Number

Register

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- (a) Analyze the CSS properties responsible for layout overlapping issues.
- (b) Examine the role of the CSS box model (margin, border, padding, content) in layout design.
- (c) Debug the layout using appropriate positioning techniques (relative, absolute, flex, grid).
- (d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- (e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Aim

To identify and fix CSS layout overlapping issues using the CSS Box Model, Flexbox, Grid, and positioning techniques.

(a) Causes of Layout Overlapping

- Fixed width and height values.
- Excessive margins and padding.
- Incorrect use of position: absolute.
- Missing box-sizing: border-box.
- Elements not wrapping on smaller screens.
- Improper use of z-index.
- Lack of responsive media queries.

(b) CSS Box Model

The CSS Box Model consists of four parts:

+-----+

| Margin |


```

| +-----+ |
| |   Border   | | | | | |
| | +-----+ | |
| | |   Padding   | | |
| | | +-----+ | | |
| | | | Content | | | |
| | | | +-----+ | | |
| | | +-----+ | |
| | +-----+ |
| +-----+
+-----+

```

- Content: Actual text or image.
- Padding: Space between content and border.
- Border: Surrounds the padding and content.
- Margin: Space outside the element.

Using box-sizing: border-box prevents unexpected element sizes.

(c) Debugging Techniques

- Relative: Positions an element relative to its normal position.
- Absolute: Positions an element relative to its positioned parent; should be used carefully.
- Flexbox: Best for one-dimensional alignment such as rows and columns.
- Grid: Best for two-dimensional layouts such as cards and page sections.

(d) Complete Working Code

Save the following as index.html.

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```
<meta charset="UTF-8">
```

```
<meta name="viewport"  
  content="width=device-width, initial-scale=1.0">
```

```
<title>CSS Layout Debugging</title>
```

```
<style>
```

```
  /* Reset and Box Model */
```

```
  * {  
    margin: 0;  
    padding: 0;  
    box-sizing: border-box;  
  }
```

```
  body {  
    font-family: Arial, sans-serif;  
    background: #f4f4f4;  
  }
```

```
  /* Header - Flexbox */
```

```
  header {  
    display: flex;  
    justify-content: space-between;  
    align-items: center;
```

```
flex-wrap: wrap;

gap: 15px;


padding: 20px 5%;
background: #222;
color: white;
}
```

```
.logo {
  font-size: 24px;
  font-weight: bold;
}
```

```
/* Navigation */
nav {
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
  gap: 20px;


  padding: 15px;
  background: white;
}
```

```
nav a {
  text-decoration: none;
```

```
    color: #222;  
    font-weight: bold;  
}
```

```
/* Main Container */
```

```
main {  
    width: 90%;  
    max-width: 1200px;  
    margin: 30px auto;  
}
```

```
/* Grid Layout */
```

```
.container {  
    display: grid;  
    grid-template-columns: 2fr 1fr;  
    gap: 20px;  
}
```

```
.content,
```

```
.sidebar {  
    background: white;  
    padding: 25px;  
    border: 1px solid #ddd;  
    border-radius: 8px;  
}
```

```
/* Relative Positioning */
```

```
.content {  
    position: relative;  
}
```

```
/* Absolute Positioning */
```

```
.badge {  
    position: absolute;  
    top: 15px;  
    right: 15px;  
  
    padding: 5px 10px;  
    background: #ddd;  
    border-radius: 5px;  
}
```

```
/* Footer */
```

```
footer {  
    display: flex;  
    justify-content: center;  
    align-items: center;  
  
    padding: 25px;  
    background: #222;  
    color: white;  
}
```

```
/* Tablet */
```

```
@media screen and (max-width: 768px) {
```

```
  .container {
```

```
    grid-template-columns: 1fr;
```

```
  }
```

```
  header {
```

```
    justify-content: center;
```

```
    text-align: center;
```

```
  }
```

```
}
```

```
/* Mobile */
```

```
@media screen and (max-width: 480px) {
```

```
  main {
```

```
    width: 95%;
```

```
    margin: 20px auto;
```

```
  }
```

```
  nav {
```

```
    flex-direction: column;
```

```
    align-items: center;
```

```
  }
```

```
.content,  
.sidebar {  
    padding: 15px;  
}
```

```
.badge {  
    position: static;  
    display: inline-block;  
    margin-bottom: 10px;  
}  
}
```

```
</style>  
</head>
```

```
<body>
```

```
<!-- Header -->
```

```
<header>
```

```
    <div class="logo">My Website</div>
```

```
    <div>Welcome User</div>
```

```
</header>
```

```
<!-- Navigation -->
```

```
<nav>
```

```
<a href="#">Home</a>
<a href="#">About</a>
<a href="#">Services</a>
<a href="#">Contact</a>
</nav>
```

```
<!-- Main Content -->
```

```
<main>
```

```
<div class="container">
```

```
<section class="content">
```

```
<span class="badge">Featured</span>
```

```
<h2>CSS Layout Debugging</h2>
```

```
<p>
```

This section demonstrates a responsive layout
using CSS Grid and relative positioning.

```
</p>
```

```
<p>
```

The layout automatically adjusts when the browser
window is resized.

```
</p>
```


</section>

<aside class="sidebar">

<h3>Sidebar</h3>

<p>

This sidebar changes its position below the
main content on smaller screens.

</p>

</aside>

</div>

</main>

<!-- Footer -->

<footer>

<p>© 2026 My Website</p>

</footer>

</body>

</html>

(e) Evaluation and Best Practices

The corrected layout prevents overlapping during browser resizing. Flexbox provides proper alignment for the header and navigation, while CSS Grid manages the main content and sidebar. The box-sizing: border-box property ensures predictable element dimensions.

Best Practices

- Use box-sizing: border-box.
- Prefer Flexbox and Grid for layouts.
- Avoid excessive absolute positioning.
- Use relative units such as %, rem, and fr.
- Add media queries for different screen sizes.
- Use gap instead of excessive margins.
- Test the webpage at different screen resolutions.

Result

The CSS layout was successfully debugged and redesigned using the CSS Box Model, Flexbox, Grid, relative positioning, absolute positioning, and media queries. The corrected webpage maintains proper alignment and prevents overlapping when the browser window is resized.

